

1. Introdução

A exploração espacial impõe desafios rigorosos à engenharia devido às condições extremas do vácuo, radiações e variações térmicas drásticas. A integridade de uma cápsula espacial e a segurança de sua tripulação dependem de sistemas eletrônicos capazes de aferir variáveis físicas e processá-las em frações de segundo. Nesse contexto, a aplicação de conceitos de Internet das Coisas (IoT) combinada com sistemas embarcados torna-se essencial para a criação de redes de telemetria robustas. Este projeto apresenta o desenvolvimento e a simulação de uma arquitetura de monitoramento em tempo real . A solução proposta não atua apenas como um registrador de dados passivo, mas como um sistema de controle ativo, avaliando flutuações de temperatura, variações de luminosidade estrutural e níveis de vibração mecânica para garantir a continuidade operacional da missão .

2. Objetivo do Projeto

O objetivo central deste trabalho é arquitetar e prototipar um sistema IoT embarcado que realize a aquisição contínua de variáveis físicas críticas para o funcionamento seguro de um módulo espacial. O sistema é projetado para ler dados brutos de sensores de temperatura, luminosidade e vibração, convertendo esses sinais elétricos em informações digitalmente processáveis . Além da simples aferição e exibição local das grandezas em um display, o projeto tem como meta secundária a implementação de rotinas lógicas de automação e segurança. Isso inclui a capacidade de o microcontrolador tomar decisões em tempo real ao identificar parâmetros fora dos limiares de segurança, acionando atuadores físicos de exaustão e emitindo alertas audiovisuais imediatos, reduzindo a dependência de comandos terrestres atrasados pela latência orbital.

3. Componentes Utilizados

Placa Microcontroladora: Arduino Uno R3, responsável pelo processamento lógico central e interfaceamento de entrada e saída.

Módulo de Exibição: Display LCD 16x2 acoplado a um módulo de comunicação I2C (chip PCF8574).

Sensores Analógicos: Um sensor de temperatura TMP36, um Fotorresistor (LDR) montado em configuração de divisor de tensão com um resistor de 10k Ω , e um Potenciômetro atuando como simulador do sensor de vibração mecânica.

Atuadores e Indicadores: Um Micro Servo Motor para simulação de válvulas físicas, um atuador piezoelétrico (Buzzer) para emissão de sinais sonoros de alerta e um LED RGB para indicação visual de status da missão.

Infraestrutura Básica: Placa de ensaio (Protoboard) e fios de ligação (Jumpers) para distribuição da malha de alimentação elétrica e roteamento dos sinais de dados.

4. Desenvolvimento do Circuito

A arquitetura elétrica do sistema foi estruturada em torno do microcontrolador, utilizando suas portas do Conversor Analógico-Digital (ADC) para a aquisição constante de sinais físicos. O sensor de temperatura, o divisor de tensão do LDR e o cursor do potenciômetro foram roteados para os pinos A0, A1 e A2, respectivamente. Para a comunicação com o display LCD, utilizou-se o barramento I2C através dos pinos SDA (A4) e SCL (A5), otimizando a pinagem do Arduino. A lógica de software desenvolvida processa esses sinais brutos e os compara com constantes de segurança predefinidas: 30 graus Celsius para o limite térmico e 75% para o limite de estresse vibracional. Caso qualquer uma dessas variáveis atinja o teto, o sistema altera seu fluxo de execução para um estado crítico. Imediatamente, sinais lógicos altos (HIGH) são enviados para o pino digital correspondente ao canal vermelho do LED RGB e para o pino do buzzer, iniciando o alarme audiovisual. Simultaneamente, um sinal PWM é enviado ao pino de controle do micro servo motor, instruindo seu eixo a girar noventa graus, o que simula a abertura mecânica de um painel de exaustão de emergência. Enquanto as variáveis permanecerem nominais, o sistema mantém o LED verde acionado e a válvula recolhida. Toda a telemetria adquirida é empacotada em uma estrutura de dados no formato JSON e enviada através da interface serial, preparando a infraestrutura para integração com gateways de comunicação de longo alcance.

5. Código Desenvolvido

O código-fonte foi escrito em linguagem C++ padrão do ecossistema Arduino, empregando bibliotecas externas essenciais como Wire.h e LiquidCrystal_I2C.h para a comunicação com o visor, além da Servo.h para a modulação por largura de pulso exigida pelo motor. Dentro do laço principal de repetição, as funções analogRead capturam a tensão dos sensores, que posteriormente sofrem tratamento matemático, como a conversão de milivolts para graus Celsius baseada no offset do componente TMP36. A estrutura condicional principal avalia o estado de perigo e bifurca as saídas digitais correspondentes. A serialização dos dados foi projetada manualmente concatenando strings para formar um objeto JSON válido.

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <Servo.h>
4
5  LiquidCrystal_I2C lcd(0x20, 16, 2);
6  Servo valvulaExaustao;
7
8  const int pinoTemp = A0;
9  const int pinoLDR = A1;
10 const int pinoVibracao = A2;
11
12 const int pinoBuzzer = 8;
13 const int pinoLedRed = 9;
14 const int pinoLedGreen = 10;
15 const int pinoServo = 7;
16
17 const float TEMP_CRITICA = 30.0;
18 const int VIB_CRITICA = 75;
19
20 void setup() {
21     Serial.begin(9600);
22     lcd.init();
23     lcd.backlight();
24
25     pinMode(pinoBuzzer, OUTPUT);
26     pinMode(pinoLedRed, OUTPUT);
27     pinMode(pinoLedGreen, OUTPUT);
28
29     valvulaExaustao.attach(pinoServo);
30     valvulaExaustao.write(0);
31
32     lcd.setCursor(0, 0);
33     lcd.print("Iniciando sist..");

```

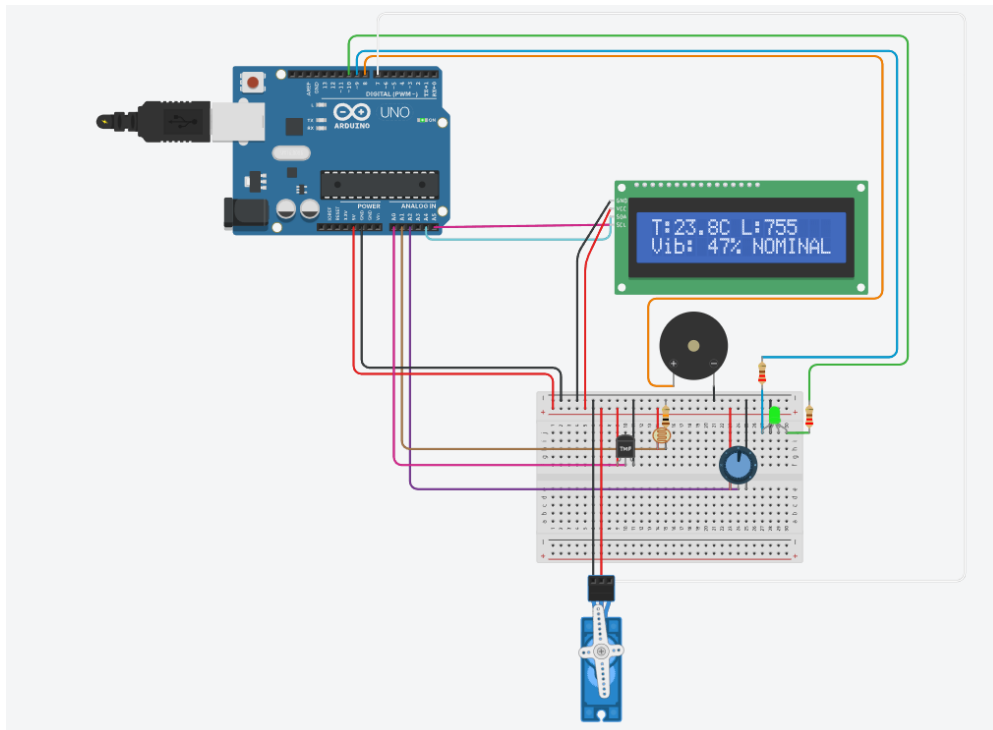
```

34     delay(2000);
35     lcd.clear();
36 }
37
38 void loop() {
39     int leituraTemp = analogRead(pinoTemp);
40     float temperaturaC = ((leituraTemp * (5.0 / 1023.0)) - 0.5) * 1
41     int leituraLDR = analogRead(pinoLDR);
42     int vibracaoPct = map(analogRead(pinoVibracao), 0, 1023, 0, 100
43
44     bool estadoCritico = false;
45
46     if (temperaturaC >= TEMP_CRITICA || vibracaoPct >= VIB_CRITICA)
47         estadoCritico = true;
48
49     digitalWrite(pinoLedGreen, LOW);
50     digitalWrite(pinoLedRed, HIGH);
51     tone(pinoBuzzer, 1000);
52
53     if (temperaturaC >= TEMP_CRITICA) {
54         valvulaExaustao.write(90);
55     }
56 } else {
57     digitalWrite(pinoLedRed, LOW);
58     digitalWrite(pinoLedGreen, HIGH);
59     noTone(pinoBuzzer);
60     valvulaExaustao.write(0);
61 }
62
63 Serial.print("{ \"temperatura\":"); Serial.print(temperaturaC);
64 Serial.print(", \"luminosidade\":"); Serial.print(leituraLDR);
65 Serial.print(", \"vibracao\":"); Serial.print(vibracaoPct);
66 Serial.print(", \"status critico\":"); Serial.print(estadoCritico)
67
68 Serial.println("}");
69
70 lcd.setCursor(0, 0);
71 lcd.print("T:");
72 lcd.print(temperaturaC, 1);
73 lcd.print("C L:");
74 lcd.print(leituraLDR);
75 lcd.print(" ");
76
77 lcd.setCursor(0, 1);
78 if (estadoCritico) {
79     lcd.print("! ALERTA CRITICO!");
80 } else {
81     lcd.print("Vib: ");
82     lcd.print(vibracaoPct);
83     lcd.print("% NOMINAL");
84 }
85
86 delay(1000);
87 }

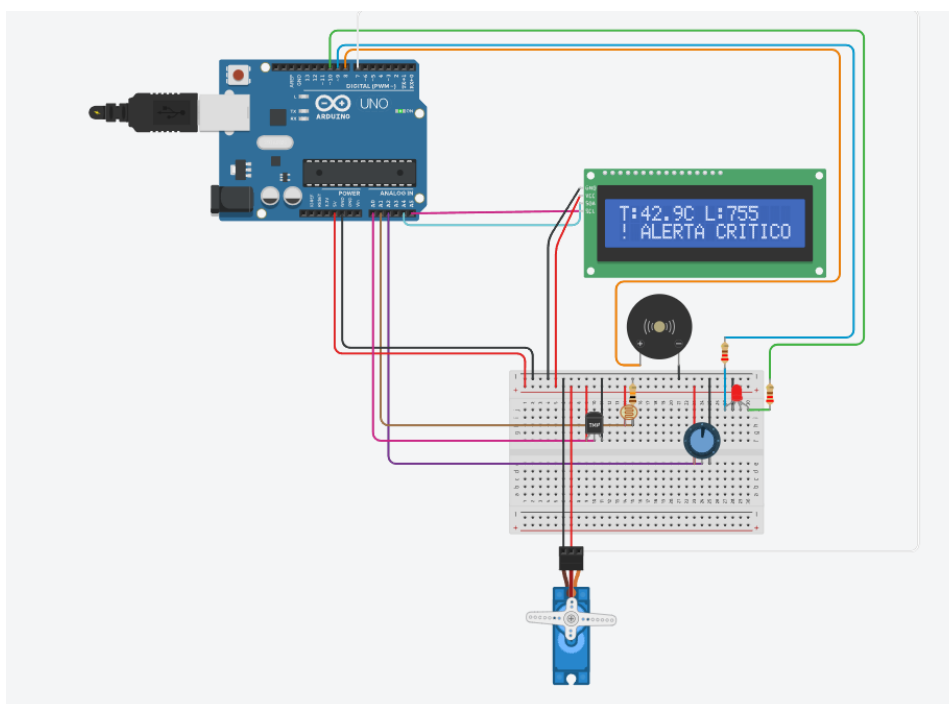
```

6. Resultados Obtidos

A execução da simulação no ambiente Tinkercad demonstrou a estabilidade do circuito elétrico e a eficiência lógica do algoritmo implementado. Em condições nominais, a comunicação I2C operou sem perdas, exibindo as variáveis em tempo real no visor LCD com a retroiluminação ativada. A formatação em JSON via monitor serial apresentou a estrutura de chave-valor correta e contínua, fundamental para eventuais sistemas de backend que venham a consumir essa telemetria. Durante o teste de estresse dos parâmetros, a simulação respondeu com latência desprezível. Ao exceder o limite térmico ou vibracional, a transição de estado foi imediata: o display reportou visualmente o estado de alerta na segunda linha, a sinalização em verde foi cortada, dando lugar ao alerta vermelho sonoro, e o servo motor realizou a varredura angular esperada de forma fluida. O restabelecimento das condições aos limiares seguros provocou a regressão automática do sistema ao estado de repouso, comprovando a resiliência do fluxo de controle desenhado. Abaixo algumas representações visuais do sistema.



Todo o circuito montado com a simulação em andamento. Com os valores nos sensores de temperatura e no potenciômetro baixos, mostrando o sistema em repouso e estável, com a LED Verde representando tudo funcionando como deveria.



O Sistema em estado de falha devido a temperatura acima dos padrões, sendo possível ver o display indicando “ALERTA CRÍTICO”, e o LED na cor vermelha, junto disso o Buzzer emitindo um som de alarme e o servo motor rotacionando 90° simulando uma abertura de válvula para resfriar o ambiente.

7. Melhorias Futuras

O protótipo atende à especificação de prova de conceito, mas a evolução para uma aplicação técnica embarcável exige o refinamento da stack de hardware e da arquitetura de software. Uma melhoria primária seria a substituição do controlador Arduino Uno de arquitetura 8-bits por uma unidade ESP32 ou microprocessador da família ARM Cortex, o que garantiria poder computacional para processar um Sistema Operacional de Tempo Real (RTOS). Isso permitiria a execução de tarefas de aferição, comunicação e atuação mecânica em threads concorrentes, eliminando os gargalos de processamento causados por funções de atraso temporal. No escopo de hardware, a simulação do atuador de vibração daria lugar a um módulo de unidade de medida inercial de seis eixos, provendo medições precisas de aceleração linear e velocidade angular espacial. Além disso, a comunicação de telemetria deixaria a dependência física do barramento serial local, passando a utilizar protocolos de rede de camada de aplicação de baixa sobrecarga como MQTT, enviando diretamente o payload de dados por conectividade de borda em órbita.

8. Conclusão

O desenvolvimento deste protótipo consolidou a aplicação prática dos conceitos de eletrônica digital, programação de baixo nível e arquitetura de dados voltados à operação em sistemas críticos de uma cápsula espacial. Através da instrumentação virtual empregada, foi possível atestar que o monitoramento em tempo real de integridade estrutural e estabilidade termodinâmica é indissociável das mecânicas de automação de resgate. O sistema construído demonstrou maturidade ao converter sinais analógicos dispersos em um bloco conciso de inteligência de máquina, saindo da esfera do monitoramento passivo para se estabelecer como um nó de decisão autônomo. Esse tipo de arquitetura lógica aproxima as simulações acadêmicas do código prático utilizado na nova indústria aeroespacial, provando que a viabilidade das missões depende da robustez preditiva e reativa embarcada em seu software.